

OBJECT ORIENTED PROGRAMMING USING JAVA

Course Code: 25CSH-238

Exam Medicine  By CU UPDATES

UNIT 1 MST-1 SMART STUDY GUIDE

A rigorous, curriculum-aligned, and technically precise examination study manual covering OOP Fundamentals, Data Types, Operators, Control Flow, Arrays, Recursion, Constructors, finalize(), and Strings. Built exclusively for first-year university candidates seeking complete syllabus coverage and conceptual mastery.

Syllabus Focus:	Unit 1 (Chapters 1.1, 1.2, and 1.3)
Target Examination:	MST-1 (Mid-Semester Test 1)
Technical Standard:	Modern Java (Modular Platform Specifications)
Author / Publisher:	Ankul kumar & Team CU Updates

TABLE OF CONTENTS

Chapter 1: Introduction to Java & OOP	
Fundamentals	Page 3
1.1 Java Characteristics, Platform Independence & Bytecode	Page 4
1.2 Accurate Explanation of JDK, JRE, and JVM Nested Mechanics	Page 4
1.3 Core Pillars of Object-Oriented Programming (Abstraction, Encapsulation, etc.)	Page 5
1.4 Dynamic Binding and Static Members	Page 6
Chapter 2: Operators, Data Types, and Flow	Control Page 7
2.1 Java Data Types: Conceptual and Representation of Primitive Sizes	Page 7
2.2 Standard Java Operators and Modulo Behavior	Page 7
2.3 Control Statements: Iteration, Conditionals, and Switch-Case Fall-Through	Page 8
2.4 Arrays, Recursive Method Stack Execution Frames & Base Case Constraints	Page 8
2.5 Java Constructors, Copy Constructors, and finalize () Method Deprecation	Page 8
Chapter 3: Java String & Core Keywords	Page 10
3.1 String Pool, String Heap Allocation, and Immutability Safeguards	Page 10
3.2 StringBuffer Synchronized Mutability and Performance Reality	Page 10
3.3 Deep Dive on Keywords: static, super, and this Constructor Chaining	Page 10

Chapter 4: Exam Preparation Kit (High-Priority Resource Deck)	Page 12
4.1 Java Syntax & Access Modifier Keyword Cheat Sheet	Page 12
4.2 Key Differences and Comparisons (String/StringBuffer, overloading/overriding, etc.)	Page 12
4.3 Comprehensive Step-by-Step Programming Solutions for MST	Page 14
4.4 High-Priority 2-Mark Short Questions & Detailed Model Answers	Page 18
4.5 High-Priority 5-Mark Long Questions & Structured Answers	Page 19
4.6 Output Tracing Questions with Detailed Step-by-Step Executions	Page 21
4.7 1-Hour Emergency Revision Sheet & Top 10 Must-Prepare Questions	Page 24

CHAPTER 1: INTRODUCTION TO JAVA & OOP FUNDAMENTALS

1.1 Java Characteristics, Platform Independence & Bytecode

Java is a high-level, platform-independent, object-oriented language developed by Sun Microsystems in 1995 [4]. At the core of Java's architecture is the philosophy 'Write Once, Run Anywhere' (WORA) [4]. This characteristic is achieved via a compilation model that separates compilation from execution. Instead of compiling source code (.java files) directly into machine-specific native instructions, the Java Compiler (javac) compiles it into an intermediate format called **Bytecode** (represented as .class files) [5, 9]. Bytecode is highly optimized, platform-neutral machine language that represents code in a format designed for a Virtual Machine rather than concrete hardware.

This intermediate Bytecode can then be transferred to any hardware system. On that machine, a **Java Virtual Machine (JVM)**—which is platform-dependent and custom-built for each specific operating system (Windows, macOS, Linux)—interprets or compiles the Bytecode at runtime into actual machine-dependent executable codes [4, 8]. Because the compiled Bytecode is standardized, any machine containing a compatible JVM can execute the program without modification, making Java profoundly portable and secure [4, 6].

PRACTICAL WORKFLOW EXAMPLE

A student compiles a program on a Windows development laptop. The .class file containing Bytecode is transferred via USB to a Linux server. The Linux server executes the program flawlessly without recompilation because the Server's Linux-native JVM reads the exact same bytecode and translates it into Linux-understandable machine instructions [5, 9].

1.2 Accurate Nesting of JDK, JRE, and JVM Mechanics

For a first-time developer, distinguishing between JVM, JRE, and JDK can be confusing. They are nested software layers [7]. The conceptual relationship is strictly defined as **JDK** $\supset$ **JRE** $\supset$ **JVM** [9]:

- 1. JVM (Java Virtual Machine):** The core execution engine [8]. Its single responsibility is loading, verifying, and executing Bytecode [8]. It also handles platform independence and automatic runtime memory management such as dynamic garbage collection [8]. The JVM does not contain the standard API standard libraries—it simply executes bytecode. It is completely abstract and platform-specific.
- 2. JRE (Java Runtime Environment):** A complete package that provides the execution environment [8]. It includes the ****JVM**** plus the **standard class libraries** (e.g., java.base, java.lang modules) and other support files that Java applications require at runtime [8]. The JRE has *no development tools*—it cannot compile a .java source code file. It is meant exclusively for end-users who only need to execute program files [8]. In modern modular Java, the runtime files are loaded dynamically from JMOD structures instead of old monolithic files like *rt.jar*.
- 3. JDK (Java Development Kit):** A complete software development environment [8]. It includes the entire ****JRE**** plus **development tools** [9]. These tools include the compiler (javac), the debugger (jdb), documentation utilities, and the compiler wrapper launcher [9]. Developers must install the JDK to write, compile, and debug Java programs [8, 9].

Software Layer	Contents	Target Audience	Compiles Source Code?
JVM	Execution engine, byte code verifier, Garbage Collector [8]	Internal engine (used by JRE) [8]	No (Only executes Bytecode) [9]
JRE	JVM + Standard Class Libraries + Configuration files [8]	End Users / Deployment servers [8]	No (Only runs programs) [9]
JDK	JRE + Compiler (javac) + Debugger (jdb) + Dev Tools [9]	Java Developers [8]	Yes (via javac compiler) [9]

COMMON MISTAKE

A common university exam mistake is claiming JRE compiles code. Remember, JRE does not have a compiler! Only JDK has the javac compiler. If you only install JRE, you can run compiled .class files, but you cannot write or compile new code.

1.3 Core Pillars of Object-Oriented Programming (OOP)

Object-Oriented Programming in Java is a paradigm that models software around real-world data and interactions [6]. Java structures code into logical containers, with the four pillars forming its core foundation [6]:

- **Class and Object Dynamics:** A **Class** is a logical blueprint or template defining the state and behaviors of a specific entity [9]. It does not allocate memory when written. An **Object** is a physical instance of that class created dynamically during runtime via the new keyword [10]. Each object has a unique identity, stored state, and behaviors defined by its class [10]. Think of a Class as a house architectural plan (occupies no physical space), while an Object is a concrete brick-and-mortar house constructed from that blueprint [10].
- **Abstraction:** The process of showing only essential, high-level features while hiding complex implementation details [11]. It answers '*what an object does*' rather than '*how it does it*' [11]. In Java, this is achieved using abstract classes and interfaces [11]. For example, when driving a car, you push the gas pedal (essential interface) without needing to understand the intricate mechanical fuel-injection combustion process occurring under the hood (implementation details) [11].
- **Encapsulation:** The process of wrapping variables (data) and methods (behavior) together into a single cohesive unit (a Class), while restricting direct access to the internal data [12]. Encapsulation is achieved by declaring instance variables as private and providing controlled access via public getter and setter methods [12]. This protects data from corruptive accidental external modification [12]. It is a data-protection mechanism [12, 13].
- **Inheritance:** A mechanism where one class (the Subclass or Child) acquires the non-private properties and methods of another class (the Superclass or Parent), promoting intensive code reusability [13]. Java implements inheritance using the extends keyword [13]. It models a logical '*IS-A*' hierarchy (e.g., a Car extends Vehicle, meaning Car *is-a* Vehicle) [13].
- **Polymorphism:** Literally means '*many forms*' [14]. It allows a single name or interface to adapt its behavior depending on the context or concrete object executing it [14]. In Java, polymorphism is divided into two distinct forms [14]:

1. **Compile-time Polymorphism (Method Overloading):** Multiple methods inside the same class share the exact same name but have different parameter lists (different number, types, or order of parameters) [14]. It is resolved by the compiler during compilation based on the method signatures.
2. **Runtime Polymorphism (Method Overriding):** A subclass provides a specific implementation for a method that is already declared in its superclass [13, 14]. This is resolved dynamically during execution using Late Binding.

CRITICAL COMPARISON

Encapsulation vs Abstraction: While closely related, they have distinct aims [13]. Encapsulation is about **data security and hiding private data** from outside access [12, 13]. Abstraction is about **hiding complex design implementation details** to simplify user interaction [11, 13]. Encapsulation is achieved via private variables and public getters/setters [12]. Abstraction is achieved via Interfaces and Abstract Classes [11].

1.4 Dynamic Binding and Static Members

Dynamic Binding (Late Binding): In method overriding, the JVM determines which method to execute during **runtime** [14, 15]. If a superclass reference variable holds a subclass object, calling an overridden method on that reference executes the subclass's version [15]. The compiler only checks if the method signature exists in the reference type, but the actual dynamic resolution at execution is based strictly on the type of the runtime object [15].

Static Members: The keyword static signifies that a member (variable or method) belongs to the **class itself** rather than to individual instances (objects) [15].

Exam Medicine 📖 By CU UPDATES

- **Static Variables:** Shared globally among all objects of that class [15]. Only one copy exists in memory (allocated when the class is loaded) [15]. If any object alters a static variable, the modification is immediately reflected across all objects [15].
- **Static Methods:** Can be called directly using the class name without instantiating any object (e.g., `Math.sqrt()`) [16]. However, static methods can *only access other static members* and static methods. They cannot access non-static instance variables or use the `this` or `super` keywords because they execute without a active object context.

CHAPTER 2: OPERATORS, DATA TYPES, & CONTROL FLOW

2.1 Java Data Types: Conceptual vs Physical Sizes

Java is a strongly-typed language, meaning every variable must be declared with a data type before compilation [18]. Java types are divided into **Primitive Types** (which store raw data values directly) and **Reference Types** (which store memory references pointing to dynamic objects on the heap) [18, 19].

A common point of confusion in university exams is the exact physical size of the boolean data type [19]. **A boolean in Java conceptually represents 1 bit of logical information (true/false)** [19]. However, the exact physical memory size of a boolean variable is **not precisely defined** by the Java Virtual Machine (JVM) Specification. In practice, the JVM compiles single boolean variables as int values (occupying 4 bytes / 32 bits) for ease of memory alignment and fast addressing in CPU registers. For arrays of booleans, the JVM represents them as byte arrays, occupying 1 byte (8 bits) per element.

Data Type	Conceptual Size	Default Value	Memory Allocation Type	Value Range / Purpose
byte	1 Byte (8 bits)	0	Primitive	-128 to 127
short	2 Bytes (16 bits)	0	Primitive	-32,768 to 32,767
int	4 Bytes (32 bits)	0	Primitive	-2^{31} to $(2^{31} - 1)$
long	8 Bytes (64 bits)	0L	Primitive	-2^{63} to $(2^{63} - 1)$
float	4 Bytes (32 bits)	0.0f	Primitive	IEEE 754 Floating point decimal
double	8 Bytes (64 bits)	0.0d	Primitive	Double-precision decimal
char	2 Bytes (16 bits)	'\u0000'	Primitive	Unicode Characters (0 to 65,535)
boolean	Logical 1-bit	false	Primitive	Stores true or false (JVM-dependent physical size)
Reference	JVM-dependent (32/64 bit)	null	Reference	Pointer to dynamic object allocated on Heap [19]

2.2 Standard Operators and Modulo Behavior with Negative Numerators

Java provides a comprehensive set of operators categorized by behavior: Arithmetic (+, -, *, /, %), Relational (>, <, ==, !=), Logical (&&, ||, !), Bitwise (&, |, ^, <<, >>), Unary (++ , --), and Assignment [20, 21].

Important Exam Concept: The Modulo Operator (%) with Negative Numbers. Many students incorrectly apply mathematical rules to Java's modulo operator. In Java, **the sign of the modulo operation is always determined by the sign of the numerator (the left operand)**. The sign of the denominator (the right operand) is completely ignored.

MUST-KNOW BEHAVIOR

- $-10 \% 3$ evaluations to **-1** (Sign comes from numerator -10)
- $10 \% -3$ evaluates to **1** (Sign comes from numerator 10)
- $-10 \% -3$ evaluates to **-1** (Sign comes from numerator -10)

2.3 Iteration, Conditional Control, and Switch Fall-Through

Java control statements alter sequential execution [21]. Conditionals like if-else and the if-else-if ladder branch code depending on a boolean test [21, 22]. Loops like for (used when iteration count is known beforehand), while (pre-tested conditional loop), and do-while (post-tested loop executing the body **at least once**) handle repetition [22, 23].

The Switch-Case and the Fall-Through Phenomenon: The switch statement evaluates an expression and jumps to the matching case branch [23]. It is cleaner than nested if statements [24]. However, execution will flow sequentially into subsequent case statements—even if their conditions do not match—unless a **break** statement is explicitly added to terminate execution [24]. This behavior is known as **Fall-Through** [24].

EXAM WARNING

Always add break; at the end of every case unless fall-through is a deliberate logic choice. Omitting it will execute the next case's statement automatically, leading to subtle logic bugs in university exams [24].

2.4 Arrays and Recursive Method Stack Frames

An **Array** is an ordered collection of homogeneous elements stored in **contiguous memory locations** [24]. In Java, arrays are reference types dynamically allocated on the heap, and they are indexed starting at 0 [19, 24].

Recursion & Stack Frame Mechanics: Recursion is a programming technique where a method calls itself directly or indirectly [25]. Every execution of a method creates an **Activation Record (Stack Frame)** in the Thread's Stack Memory. This stack frame contains the method's local variables, parameters, and return address. When a method calls itself, a new stack frame is pushed on top of the current stack [26]. When the base case is reached, the frame returns its result and is popped, flowing back through the execution stack.

The Base Case Constraint: A recursive method must have a **Base Case** that halts self-invocation, and a **Recursive Case** that drives the input closer to the base case [25]. If the base case is missing, recursive execution proceeds infinitely. This rapidly fills the thread's Stack Memory with active frames, leading to a catastrophic **java.lang.StackOverflowError** and crashing the application [26].

2.5 Java Constructors, Copy Constructors, and the finalize() Deprecation

Constructors: Special class members executed automatically during object instantiations via new [27]. They have no return type and their name matches the Class exactly [27].

- **Default Constructor:** Inserted automatically by the Java Compiler *only if* the developer writes no explicit constructors. It accepts no arguments and initializes all fields to default values (0, null, false).
- **Parameterized Constructor:** User-defined constructor accepting specific parameters to initialize instance state with custom user values [27, 28].

Copy Constructor: Explicit constructor that accepts an object of the same class and copies its values [28]. Unlike C++, Java does not provide an automatic copy constructor. It must be written explicitly [28, 29]. Page 9

The finalize() Method Reality: In early Java versions, the finalize() method was invoked by the Garbage Collector immediately before destroying an object to perform resource cleanup [29]. **However, in modern Java, the finalize() method is officially DEPRECATED (since Java 9) and should never be used.** The JVM does not guarantee when (or even if) finalize() will execute, as garbage collection runs asynchronously and unpredictable [29]. Relying on it causes memory leaks and performance issues [29]. Standard resource management is handled via the java.lang.AutoCloseable interface combined with a try-with-resources block [29, 30].

EXAM-ESSENTIAL CLARIFICATION

final vs finally vs finalize(): This is a classic 5-Mark question! [30]

- **final:** A modifier keyword. Used to define constants, prevent method overriding, and block class inheritance [30].
- **finally:** A control block used in exception handling. It guarantees execution of cleanup code, regardless of whether an exception was thrown or caught [30].
- **finalize():** A deprecated method of the Object class, called before garbage collection. Unreliable and obsolete [29, 30].

CHAPTER 3: JAVA STRING & CORE KEYWORDS

3.1 String Pool, Heap Allocation, and Immutability Mechanics

In Java, a **String** is a reference object representing an immutable sequence of characters, managed by the predefined `java.lang.String` class [32]. There are two distinct mechanisms for creating Strings, which affect memory layout [32]:

1. **Using String Literals:** (e.g., `String s = "Java";`) [32]. The JVM checks the **String Constant Pool (SCP)**, a specialized cache area residing inside the Heap [32, 33]. If the literal 'Java' is already in the pool, the JVM simply returns a reference to that existing instance without allocating new memory [32]. If it is not present, a new literal object is created in the SCP [32]. This allows efficient memory sharing.
2. **Using the new Keyword:** (e.g., `String s = new String("Java");`) [32]. The JVM bypasses SCP reuse and forcefully allocates a completely new, distinct String object in the **general Heap memory** [32, 33]. This object points to the internal character value, but it is a separate physical object [32]. This can result in redundant memory allocation.

The Logic of String Immutability: Once created, the content of a String object cannot be altered [33]. Any operation that appears to modify a String (like concatenation with `+` or `concat()`) actually creates a completely new String object with the combined characters, while leaving the original object unchanged [33]. This is implemented by declaring the backing array (character array) as `private` and `final`, with no setter methods.

Immutability is critical for: **Security** (crucial parameters like database URLs or passwords cannot be modified by malicious code), **String Pool efficiency** (safe sharing of identical literals), **Thread Safety** (multiple threads can read the same String without synchronization risks), and **Hashing** (hashcode is cached during creation, making Strings fast keys for HashMaps) [33].

VITAL SYNTAX WARNING

The String Reference Trap: Evaluating `s1 == s2` compares their **memory addresses**, while `s1.equals(s2)` compares the **actual character values** [34]. For literals (SCP), `==` returns `true`. For heap objects created via `new`, `==` returns `false`, even if both contain identical text! Always use `.equals()` for value comparison [34].

3.2 StringBuffer: Synchronized Mutability and Performance

StringBuffer: To avoid creating numerous temporary String objects during frequent modifications (like in long loops), Java provides the `StringBuffer` class [35]. `StringBuffer` represents a **mutable sequence of characters** [35]. Its contents can be appended, inserted, or reversed in-place without creating new objects [35, 36].

The Performance Balance: In terms of thread safety, `StringBuffer` is completely **synchronized (thread-safe)** [40]. Its methods are guarded by synchronization locks, ensuring multiple threads can write to it safely [40]. However, this synchronization introduces slight runtime overhead. For single-threaded loops, modern Java developers prefer `StringBuilder` (which is identical to `StringBuffer` but non-synchronized, yielding faster execution), though `StringBuffer` remains the key mutable sequence on your syllabus [40].

3.3 Keyword Deep Dive: static, super, and this Constructor Chaining

Java provides keyword mechanisms to control access, hierarchy, and context [37, 38]:

this keyword: Represents the current class instance reference [38]. It is primarily used to resolve naming conflicts when instance variables and parameters share identical names (e.g., `this.rollNo = rollNo;`) [39, 40]. It can also be used to perform **Constructor Chaining** by calling another constructor in the same class using `this()` [39]. Page 11

super keyword: Represents the immediate parent class instance reference [37, 38]. It is used to access parent variables (`super.field`), invoke overridden parent methods (`super.method()`), and call the parent constructor (`super()`) [38].

Strict Constructor Chaining Rules: When chaining constructors, the call to `this()` or `super()` **must be the absolute first statement inside the constructor body** [38]. Because of this rule, **you can never call both `this()` and `super()` in the same constructor**. Doing so will cause an immediate compiler error. If a constructor contains no explicit call to `this()` or `super()`, the compiler automatically inserts an implicit call to `super()` (no-argument) as the first statement to ensure the parent class is initialized first [38].

CU UPDATES

CHAPTER 4: COMPREHENSIVE EXAM PREPARATION KIT

4.1 Java Syntax & Access Modifier Keyword Cheat Sheet

Understanding access scopes is crucial for encapsulation. This table details where variables and methods are accessible based on their modifier keywords.

Modifier Key	Same Class?	Same Package Subclass?	Same Package Non-Subclass?	Different Package Subclass?	Global Different Package ?
private	Yes (Strictly Class)	No	No	No	No
default (No keyword)	Yes	Yes	Yes	No	No
protected	Yes	Yes	Yes	Yes (via inheritance)	No
public	Yes	Yes	Yes	Yes	Yes (Any where)

4.2 Core Difference and Comparison Matrices

I. String vs. StringBuffer [40]

Feature / Property	String Class [40]	StringBuffer Class [40]
Mutability [40]	Immutable. Contents cannot be modified once created [33].	Mutable. Contents can be modified in-place [35].
Memory Allocation [40]	Pool (SCP) & Heap. Sharing saves memory [32, 33].	Allocated directly in standard Heap memory [35].
Performance [40]	Slower for repetitive operations due to constant object creation.	Faster for repetitive operations; modifies the same buffer [35, 36].
Thread Safety [40]	Yes. Naturally thread-safe due to immutability [33].	Yes. Synchronized methods ensure thread-safe access [40].

II. Method Overloading vs. Method Overriding

Feature / Aspect	Method Overloading (Compile-time) [14]	Method Overriding (Runtime) [14]
Class Context	Occurs within a single class.	Occurs between a Superclass and a Subclass [13, 14].
Method Name	Must be identical.	Must be identical.
Parameters	Must be different (count, type, or order of parameters).	Must be identical in count, type, and order.
Return Type	Can be different (does not affect overloading).	Must be identical or covariant (subclass of parent return).
Binding Mode	Static / Early binding (resolved by compiler at compile-time).	Dynamic / Late binding (resolved by JVM at runtime) [14, 15].

III. this vs. super Keyword [41]

Aspect	this Keyword [41]	super Keyword [41]
Reference target [41]	Points to the current class instance.	Points to the immediate parent class instance [38].
Variables Access [41]	Accesses current class instance variables [39].	Accesses immediate parent class variables [38].
Method Calls [41]	Invokes current class methods.	Invokes overridden parent class methods [38].
Constructor Chain [41]	Calls overloaded constructor inside same class: this() [39].	Calls parent constructor: super() [38].

4.3 Step-by-Step Programming Solutions

Program 1: Complete Class, Parameterized and Copy Constructors

```
class Student { private
int rollNo; private
String name;

    // Parameterized Constructor public Student
    (int rollNo, String name) { this.rollNo =
    rollNo; this.name = name;

    }

    // Explicit Copy Constructor
    public Student(Student other) {
        this.rollNo = other.rollNo; // Copying data fields
        this.name = other.name;
    }

    public void display() {
        System.out.println("Roll No: " + rollNo + ", Name: " + name);
    }
}

public class Main { public static void main
(String[] args) {
    Student s1 = new Student(101, "Rahul"); Student s2 = new
    Student(s1); // Invoking copy constructor

    s1.display();
    s2.display();
}
}
```

Execution Trace / Logic: Class Student defines instance fields rollNo and name. The first parameterized constructor initializes fields using the current object reference (this.rollNo = rollNo). The second constructor is an explicit **Copy Constructor** that accepts a reference to another Student object (other) and manually duplicates its field values [28]. The Main method creates s1, then creates s2 by passing s1. The fields are cloned perfectly [28].

Output:

Roll No: 101, Name: Rahul

Roll No: 101, Name: Rahul

Program 2: Inheritance, Method Overriding, super and Dynamic Binding

```
class Vehicle {
    String type = "Generic Vehicle";

    public void displayInfo() {
        System.out.println("This is a: " + type);
    }
}

class Car extends Vehicle {
    String type = "Luxury Sedan";

    @Override public void displayInfo() { super.displayInfo(); //
        Invoking parent class method System.out.println("Specifically, this
        is a: " + this.type); System.out.println("Parent class type was: " +
        super.type);
    }
}

public class Main { public static void main
(String[] args) {
    Vehicle v = new Car(); // Parent reference, child object
    v.displayInfo(); // Dynamic Binding resolves to Car's method
}
}
```

Execution Trace / Logic: Car inherits Vehicle. It overrides the displayInfo() method. Inside the subclass method, super.displayInfo() is called, executing the parent class's behavior [38]. Then, this.type and super.type are utilized to access child and parent variables separately, illustrating that variable hiding is resolved at compile time whereas method execution is resolved dynamically [38]. In main(), the reference variable is of type Vehicle but holds a concrete Car object. When v.displayInfo() is executed, the JVM dynamically binds the call to the subclass's overridden method [14, 15].

Output:

This is a: Generic Vehicle

Specifically, this is a: Luxury Sedan

Parent class type was: Generic Vehicle

Program 3: Contiguous Memory Array Traversal (Maximum & Average)

```
public class ArrayDemo { public static void
main(String[] args) { int[] marks = {75, 80,
90, 88, 92};
    int max = marks[0];
    int sum = 0;

    for(int i = 0; i < marks.length; i++) {
        sum += marks[i]; if(marks[i] > max) { max
        = marks[i];

    } } double average = (double) sum /
    marks.length;

    System.out.println("Max element: " + max);
    System.out.println("Average: " + average);
}
}
```

Execution Trace / Logic: An integer array marks is allocated in contiguous heap memory [24]. The program initializes max with the first element (index 0) and iterates through the loop [25]. During each iteration, it updates the running sum and performs a comparison to update max. Finally, it casts sum to a double before division to prevent integer division truncation.

Output:

Max element: 92

Average: 85.0

Program 4: Recursive Factorial Calculation with Stack Frame Trace

```
public class FactorialDemo { public static int factorial
(int n) { if (n == 1) { // Base Case return 1; } return n
* factorial(n - 1); // Recursive Case

}

public static void main(String[] args) {
    System.out.println("5! = " + factorial(5));
}
}
```

Execution Stack Trace for factorial(5):

1. main() invokes factorial(5). Stack frame f(5) is pushed. (Checks 5==1: False). Needs 5 * f(4).
2. f(4) pushed. Needs 4 * f(3).
3. f(3) pushed. Needs 3 * f(2).
4. f(2) pushed. Needs 2 * f(1).
5. f(1) pushed. Base Case matches (1==1: True). Returns 1 [25, 26].
6. Frames pop in reverse order [26]:
 - f(2) computes 2 * f(1) = 2 * 1 = 2, returns 2 and pops.
 - f(3) computes 3 * f(2) = 3 * 2 = 6, returns 6 and pops.

- $f(4)$ computes $4 * f(3) = 4 * 6 = 24$, returns 24 and pops.
- $f(5)$ computes $5 * f(4) = 5 * 24 = 120$, returns 120 and pops [26].

7. `main()` prints 120.

Output:

$5! = 120$

CU UPDATES

4.4 High-Priority 2-Mark Short Questions & Model Answers

Q1: Explain why the JVM is considered platform-dependent while Java is platform-independent. [42]

A1: Java is platform-independent because its compiler compiles source code into standardized, platform-neutral **Bytecode** (.class files) [5, 6]. However, the JVM is platform-dependent because it must be specifically engineered for each target operating system to translate that uniform Bytecode into the actual native machine instructions of the local CPU [8]. Hence, platform-independent bytecode runs on platform-dependent JVMs [4, 5].

Q2: What is a Copy Constructor? How is it implemented in Java? [28]

A2: A Copy Constructor is a constructor that initializes a new object using an existing object of the same class [28]. Unlike C++, Java does not provide an automatic, built-in copy constructor [28, 29]. It must be written manually by defining a constructor that accepts a reference to the class itself as a parameter, then copying the fields (e.g., `this.x = other.x`) [28].

Q3: Why is String immutability critical in Java? State two reasons. [33]

A3: String immutability is critical because:

1. **Security:** Sensitive inputs (passwords, database connection parameters, network sockets) are protected from accidental or malicious modification [33].
2. **String Pool Efficiency:** It allows the JVM to share a single String literal across multiple reference variables, saving massive heap memory [33].

Q4: What is the purpose of the static keyword when applied to methods? [36]

A4: Applying static to a method makes it belong to the class itself rather than instances [36]. It can be invoked directly via the class name without creating an object (e.g., `Math.sqrt()`) [36]. However, static methods can only access static members and cannot refer to non-static variables, methods, or utilize `this` or `super`.

Q5: What is a StackOverflowError? When does it occur? [26]

A5: A `StackOverflowError` is a runtime error that occurs when the Thread Stack memory is completely exhausted [26]. This is typically caused by infinite recursion, where a method calls itself repeatedly without meeting a terminating **Base Case**, causing the JVM to run out of memory for stacking execution frames [26].

Q6: What is the physical size of a boolean in Java? Discuss. [19]

A6: Conceptually, a boolean represents 1 bit of logical information [19]. However, its exact physical memory size is **not defined** by the JVM Specification. In actual execution, the JVM compiles individual booleans to 32-bit int variables for faster processing, while arrays of booleans are stored as 8-bit byte arrays.

4.5 High-Priority 5-Mark Long Questions & Model Answers

Q1: Describe the complete execution process of a Java program, detailing the exact roles of the JDK, Compiler, Bytecode, JRE, and JVM. Use a text-flow description to map their integration. [45]

A1: Execution Architecture Process:

- 1. Writing Source Code:** The developer writes human-readable Java source code and saves it as a `.java` file [9].
- 2. Compilation (JDK Layer):** The developer runs the Java Compiler (`javac`), which is a key component of the **JDK** [9]. The compiler parses the source code. If no syntax errors are present, it translates the source code into platform-neutral **Bytecode**, saving it as a `.class` file [5, 9].
- 3. Loading and Runtime JRE Provisioning:** To execute, the compiled `.class` file is handled by the **JRE** [9]. The JRE provides the necessary runtime libraries (standard Java modules) and initializes the JVM environment [8, 9].
- 4. JVM Execution Mechanics:** Inside the JVM, three consecutive subsystems execute the bytecode [8]:
 - **Class Loader:** Dynamically loads the required class files into the JVM's memory areas.
 - **Bytecode Verifier:** Screens the bytecode for security violations (such as direct memory access or corrupt code) before allowing execution [6].
 - **Execution Engine:** Translates bytecode into native binary machine instructions [8]. It uses an **Interpreter** (translates line-by-line) and a **Just-In-Time (JIT) Compiler** [7]. The JIT compiler monitors code execution and compiles 'hot' spots of frequently repeated bytecode into native machine code directly, greatly enhancing execution performance [7].

Thus, Java achieves physical compilation to intermediate bytecode, and runtime execution/interpretation via OS-specific JVMs.

Q2: Define and compare constructors in Java. Discuss the Default Constructor, Parameterized Constructor, and Programmer-defined Copy Constructor. Explain when each is preferred. [47]

A2: Constructor Analysis:

A Constructor is a special block of code that initializes a newly instantiated object. It bears the identical name of the class and lacks any return type [27].

- **Default Constructor:** If a programmer declares absolutely no constructors, the Java compiler automatically synthesizes an implicit no-argument constructor [30]. It sets all member variables to default states (numerical types to 0/0.0, booleans to false, reference types to null). *When Preferred:* When default initialization is sufficient, or when using framework reflection APIs that require blank instantiation [2]. Once a custom constructor is defined, the default constructor is lost unless rewritten explicitly.
- **Parameterized Constructor:** A custom constructor designed to accept arguments [27]. It is used to initialize the state of an object with custom, programmer-supplied values during object creation [27, 28]. *When Preferred:* Standard practice for object-oriented development. It enforces that an object cannot exist in an uninitialized or blank state.
- **Copy Constructor:** A constructor that takes an existing object of the same class as a parameter to duplicate its current values [28]. Java has no default copy constructor; it must be manually implemented [28, 29]. *When Preferred:* When you need to create a copy of an object while avoiding copying memory references. It provides a clean, manual alternative to the `Object` class's `clone()` method.

Q3: What is the finalize() method? Explain its execution mechanics, why it is deprecated in modern Java, and describe the modern robust alternative for resource management. [29]

A3: finalize() Method Evaluation:

- **Original Purpose:** The finalize() method is a protected method defined in the java.lang.Object class [29]. In legacy Java, the Garbage Collector (GC) would invoke finalize() immediately before destroying an object to let the programmer release system resources (like file handles or database connections) [29].
- **Why it is Deprecated (Java 9+):** finalize() is fundamentally flawed and deprecated because:
 1. **No Execution Guarantee:** The GC operates asynchronously. The JVM does not guarantee *when* GC will run, meaning finalize() might never execute before the program terminates [29].
 2. **Severe Performance Hit:** Objects with finalize() registered must be queued in a finalize queue, delaying garbage collection and causing memory leaks.
 3. **Security Risk:** If an exception occurs inside finalize(), execution halts, leaving the object half-destroyed. This can be exploited for finalize resurrection attacks.
- **Modern Robust Alternative:** The standard approach is implementing the **java.lang.AutoCloseable** interface [29, 30]. Classes override the close() method. The resource is managed using a **try-with-resources** block, which guarantees the close() method executes automatically upon block exit, even if exceptions are thrown [29, 30].

4.6 Output Tracing Questions

Question 1: Unary Increment Operators

```
public class UnaryTrace { public static void main(String
[] args) { int x = 5; int y = ++x * x++;
System.out.println("x = " + x + ", y = " + y);

    }
}
```

Step-by-Step Execution Trace:

1. Variable x is initialized to 5.
2. The expression evaluates `++x * x++` left-to-right:
 - `++x` is a **pre-increment**. x increases to 6 first, and the operand evaluates to 6.
 - Next, `x++` is a **post-increment**. The operand evaluates to the current value of x (which is 6), and then x is incremented to 7.
3. The multiplication is performed: $6 * 6 = 36$.
4. y is assigned 36, and x ends with the value 7.

Output: x = 7, y = 36

Question 2: String Pool Allocation and == vs equals()

```
public class StringTrace { public static void
main(String[] args) {
    String s1 = "Java"; String s2 =
    new String("Java"); String s3 =
    s2.intern(); String s4 = "Java";

    System.out.println(s1 == s2);
    System.out.println(s1 == s3);
    System.out.println(s1 == s4);
    System.out.println(s1.equals(s2));
}
}
```

Step-by-Step Execution Trace:

1. s1 is created as a literal. The JVM checks the String Constant Pool (SCP). It creates a 'Java' object in the SCP and refers s1 to it [32].
2. s2 is created using `new String()`. This allocates a completely separate object in the Heap [32].
3. `s2.intern()` returns the canonical representation. It looks for 'Java' in the SCP, finds it (created by s1), and returns its reference. Thus, s3 points directly to the SCP object of s1 [32, 33].
4. s4 is a literal. It reuses the SCP reference of s1 [32].
5. Evaluations:
 - `s1 == s2`: Comparing references. SCP object vs Heap object → **false** [34].
 - `s1 == s3`: Comparing references. SCP object vs SCP object → **true** [34].

- s1 == s4: Comparing references. SCP object vs SCP object → **true** [34].
- s1.equals(s2): Comparing values. 'Java' value matches 'Java' value → **true** [34].

Output:*false**true**true**true***Question 3: Dynamic Binding and Overriding**

```

class Parent { public void
print() {
    System.out.println("Parent print");
} } class Child extends Parent
{

    @Override public void
    print() {
        System.out.println("Child print");
    } } public class OverridingTrace { public
static void main(String[] args) {

    Parent obj1 = new Parent();
    Parent obj2 = new Child();

    obj1.print();
    obj2.print();

    }
}

```

Step-by-Step Execution Trace:

1. obj1 is declared as Parent reference and instantiated as Parent object. Calling print() binds statically to the Parent's method → prints Parent print.
2. obj2 is declared as Parent reference but instantiated as a Child object. At compile-time, the compiler verifies that Parent has a print() method. At runtime, the JVM uses Dynamic Binding. It checks the actual object type, which is Child, and executes the Child's overridden method → prints Child print [14, 15].

Output:*Parent print**Child print*

Question 4: Switch-Case Fall-Through

```
public class SwitchTrace { public static void
main(String[] args) { int value = 2; switch
(value) { case 1:

        System.out.print("One "); case
        2:
        System.out.print("Two "); case
        3:
        System.out.print("Three ");
        break; default:

        System.out.print("Default ");
    }
}
```

Step-by-Step Execution Trace:

1. value is 2. The switch jumps to case 2: [23].
2. It executes System.out.print("Two ").
3. Crucially, case 2 has no break; statement [24]. This triggers **Fall-Through**. Execution flows directly into case 3 statements without checking the condition [24].
4. It executes System.out.print("Three ") inside case 3.
5. case 3 contains a break; statement, which terminates the switch execution and exits the block [24].

Output: Two Three

4.7 1-Hour MST-1 Emergency Revision Sheet

Platform Independence Formula: Source (.java) --[javac]--> Bytecode (.class) --[JVM]--> Machine Code (OS-specific) [5, 9].

□ **Nested Hierarchy:** JDK (Dev tools + Compiler) = JRE (Libraries) + JVM (Execution Engine) [9].

Page 24

JVM Memory (Conceptual): **Stack** handles per-thread method activation frames and local variables [26]. **Heap** is shared globally and stores dynamic objects [19, 32]. **Metaspace** handles class structure meta-data [2].

boolean physical size: Not fixed. Conceptually represents 1 logical bit of info. In JVM execution, it consumes 32 bits (int representation) or 8 bits in byte-arrays for CPU register alignments [19].

Java Modulo Operator (%): Sign of the modulo evaluation is determined solely by the sign of the **numerator** (the left operand). Denominator's sign is completely ignored.

Switch Fall-Through: Omitting break; statement causes sequential execution to flow directly into the next case block regardless of condition matching [24].

Recursion Stack Overflow: Missing **Base Case** causes infinite execution frames to fill Thread Stack Memory, throwing java.lang.StackOverflowError [26].

Constructor Chaining Rule: this() (current constructor call) or super() (parent constructor call) must be the **very first statement** in a constructor. You can **never** call both in the same constructor [38].

String Constant Pool (SCP): Resides in the heap area [33]. Literally created strings reuse pool objects. Strings created via new bypass literal pooling and allocate distinct objects directly in Heap memory [32].

finalize() Deprecation: Since Java 9, finalize() is deprecated. It is unpredictable and slow. Manage resources using java.lang.AutoCloseable interface and standard try-with-resources blocks [29, 30].

TOP 10 MUST-PREPARE QUESTIONS FOR MST-1 SUCCESS

- 1. Explain platform-independence in Java with the roles of Bytecode and JVM. [42]
- 2. Differentiate JDK vs JRE vs JVM with a nesting representation and comparison matrix. [42]
- 3. Define the 4 pillars of OOP with clear examples and technical implementations. [46]
- 4. Explain Method Overriding and Runtime Polymorphism using the concept of Dynamic Binding. [46]
- 5. Discuss primitive data types, highlighting the boolean physical size and allocation. [19]
- 6. Discuss modulo behavior with negative numbers and switch fall-through phenomenon. [44]
- 7. Explain stack frame mechanics in recursion and why base cases prevent StackOverflowErrors. [47]
- 8. Compare default, parameterized, and programmer-defined copy constructors in Java. [47]
- 9. Explain String immutability, the String Constant Pool (SCP) and why it is used. [48]
- 10. Differentiate String and StringBuffer on memory, performance, and thread-safety. [48]